Interview Q&A 2

Software Engineer / iGaming – Dabble

Technical Questions

Q1. Tell me about your software development experience.

Answer:
My experience currently comes mainly from university and practice projects. I have worked with C#, JavaScript, TypeScript, Python and SQL, along with React, Vue.js, Angular, ASP.NET Core, Entity Framework Core and SQL Server. My largest practical project was a full-stack Task Manager where I created a REST API and connected three different frontend frameworks to the same backend.

Q2. What is the difference between JavaScript and TypeScript?

Answer:
JavaScript is a dynamically typed programming language, while TypeScript extends JavaScript with static typing and additional development features. TypeScript can identify many type-related problems during development rather than waiting until runtime. I used TypeScript with React, Vue.js and Angular in my frontend practicals.

Q3. Compare React, Vue.js and Angular based on your experience.

Answer:
All three can be used to create component-based frontend applications, but they approach development differently. React uses JSX and hooks such as useState and useEffect. Vue uses features such as ref, v-model, v-for and onMounted. Angular provides a more structured framework with dependency injection, services such as HttpClient, data binding and lifecycle features such as ngOnInit. Building the same Task Manager in all three helped me see those differences practically.

Q4. Explain GET, POST, PUT and DELETE.

Answer:
GET retrieves data, POST creates new data, PUT updates existing data and DELETE removes data. My Task Manager API implemented all four operations, and I tested them using Postman before connecting the frontend.

Q5. Why use Entity Framework Core instead of writing SQL for every operation?

Answer:
Entity Framework Core allows the application to work with database records through C# objects and provides an abstraction over many common database operations. It can make application code easier to maintain and reduces repetitive SQL. However, I think understanding SQL is still important, which is why I first built a version using direct SQL Server access before moving to Entity Framework Core.

Q6. What is dependency injection?

Answer:
Dependency injection provides a class with the dependencies it needs rather than having the class create them itself. I encountered this in ASP.NET Core when injecting the database context into my controller, and in Angular when using services such as HttpClient. It helps separate responsibilities and makes applications easier to maintain and test.

Q7. What happens when your frontend sends a request to your Task Manager API?

Answer:
The frontend sends an HTTP request to an ASP.NET Core endpoint. The controller processes that request and uses Entity Framework Core to interact with SQL Server. The database operation is performed and the API returns an HTTP response, usually containing JSON data. The frontend then updates what the user sees based on the response.

Q8. How would you debug a full-stack application?

Answer:
I would isolate each layer. I would verify the database and backend first, test the API independently using Postman, and then inspect the frontend request through browser DevTools. I would check the HTTP status code, request and response data, console errors, API logs, CORS settings and database behaviour. This prevents me from assuming the frontend is responsible when the problem may be somewhere else.

Q9. Why are WebSockets relevant to applications with live information?

Answer:
WebSockets allow the server and client to maintain a persistent two-way connection. That makes them useful when information needs to be delivered to users quickly

without repeatedly polling the server. Examples include live scores, notifications, chat and other real-time application features.

Q10. What is asynchronous programming and where have you encountered it?

Answer:
Asynchronous programming allows an application to wait for operations such as network or database requests without unnecessarily blocking other work. I used async and await when working with API calls and Entity Framework Core methods such as ToListAsync, FindAsync and SaveChangesAsync.

Q11. Why did you test your API with Postman before connecting the frontend?

Answer:
It allowed me to verify that the backend worked independently. If GET, POST, PUT and DELETE worked correctly in Postman but failed in the frontend, I knew the problem was more likely related to frontend code, the request configuration or CORS. It made troubleshooting much more systematic.

Q12. What role does Git play in software development?

Answer:
Git provides version control, allowing developers to track changes and maintain the history of a project. I used Git and GitHub throughout my academic work to organise source code and evidence. I also learned to exclude generated files such as node_modules, bin, obj and dist rather than committing unnecessary project files.

Behavioural Questions – Software Engineer / iGaming

Q13. Tell me about a project you're proud of.

Answer:
My full-stack Task Manager is the project that best represents my progress. It started with C# and direct SQL Server access. I then created an ASP.NET Core REST API, introduced Entity Framework Core for persistent database access and built React, Vue.js and Angular frontends against the same API. It helped me understand how the different parts of a full-stack system work together.

Q14. Tell me about a difficult bug you solved.

Answer:
One example was an Angular application that displayed a blank page and produced an NG0908 error. I inspected the runtime error and discovered that the project was configured for modern zoneless Angular while part of my configuration expected Zone.js. I corrected the application configuration and retested it successfully. The experience reinforced the importance of reading errors carefully rather than immediately rewriting code.

Q15. How do you ensure the code you write works correctly?

Answer:
At my current level, I test functionality incrementally. For example, I tested each REST endpoint independently with Postman before integrating the frontend. Then I tested actions such as adding, completing and deleting tasks through the UI and verified that the database reflected those operations. As I continue developing, I also want to expand my experience with automated unit and integration testing.

Q16. How do you deal with unfamiliar technology?

Answer:
I start with the fundamentals and then build something small with it. During my program I moved between React, Vue.js and Angular rather than only reading about their differences. Implementing the same application in each framework made the concepts much easier to understand and gave me practical evidence of what I had learned.

Q17. Why are you interested in software engineering in an iGaming environment?

Answer:
I am interested in software that involves interactive user experiences, APIs, data and potentially real-time functionality. My learning in TypeScript, frontend frameworks, ASP.NET Core, databases and WebSockets gives me a foundation in several of those areas. I would like to build on that foundation in a professional engineering environment where performance, reliability and user experience matter.

Q18. You are applying toward a senior-level role, but you're currently a student. How would you address that?

Answer:
I would be transparent that I am not claiming senior-level professional experience. I

selected the role as a target for my career preparation because its technology requirements give me a useful direction for developing my skills. My current evidence comes from academic and practice projects, and my goal is to continue building the professional experience needed to progress toward that level.